

EXPERIMENT-17

1. Algorithm:

1. Start.
2. Declare integer variable choice.
3. Display the menu:

Generate the largest possible number using its digits.

Generate the smallest possible number using its digits.

Swap its first and last digits.

Check whether it is a palindrome.

4. Read choice.
5. Use switch(choice):

If choice = 1, call largestNumber().

If choice = 2, call smallestNumber().

If choice = 3, call swapFirstLast().

If choice = 4, call checkPalindrome().

Otherwise, display "Invalid choice".

6. Stop.

Algorithm for largestNumber():

1. Start.
2. Input a positive integer n.
3. Create an array freq[10] initialized to 0.
4. Extract each digit using % 10 and increase its frequency.
5. Arrange the digits from 9 down to 0.
6. Form the new number using these digits.

7. Display the largest possible number.
8. Stop.

Algorithm for smallestNumber():

1. Start.
2. Input a positive integer n.
3. Count the frequency of every digit from 0 to 9.
4. Find the smallest non-zero digit and place it first.
5. Place all zeroes immediately after the first digit.
6. Place the remaining digits in ascending order.
7. Display the smallest possible number.
8. Stop.

Note: A zero cannot be placed first because it would become a leading zero. For example, using the digits of 31042, the smallest number is 10234, not 01234.

Algorithm for swapFirstLast():

1. Start.
2. Input a positive integer n.
3. If n contains only one digit, display the same number.
4. Find the last digit using $n \% 10$.
5. Find the appropriate divisor (10, 100, 1000, etc.) to obtain the first digit.
6. Find the first digit using $n / \text{divisor}$.
7. Find the middle portion of the number.
8. Construct the new number by placing the last digit first and the first digit last.
9. Display the result.
10. Stop.

Algorithm for checkPalindrome():

1. Start.
2. Input a positive integer n.
3. Store n in temp.
4. Initialize reverse = 0.
5. Extract each digit of temp.
6. Add the digit to reverse in reverse order.
7. Compare reverse with the original number n.
8. If both are equal, display that the number is a palindrome.
9. Otherwise, display that the number is not a palindrome.
10. Stop.

2. Test Case Table:

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	31042	Largest = 43210	Largest = 43210	Pass
2	1	52731	Largest = 75321	Largest = 75321	Pass
3	2	31042	Smallest = 10234	Smallest = 10234	Pass
4	2	54321	Smallest = 12345	Smallest = 12345	Pass
5	3	12345	After swapping = 52341	After swapping = 52341	Pass
6	3	1234	After swapping = 4231	After swapping = 4231	Pass
7	4	12321	Palindrome	Palindrome	Pass
8	4	12345	Not Palindrome	Not Palindrome	Pass

3. Program, Compile and Execution:

```
#include <stdio.h>

void largestNumber(void);
void smallestNumber(void);
void swapFirstLast(void);
void checkPalindrome(void);

int main(void)
{
    int choice;

    printf("—— MENU ——\n");
    printf("1. Generate largest possible number\n");
    printf("2. Generate smallest possible number\n");
    printf("3. Swap first and last digits\n");
    printf("4. Check palindrome\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            largestNumber();
            break;

        case 2:
            smallestNumber();
            break;

        case 3:
            swapFirstLast();
            break;

        case 4:
            checkPalindrome();
            break;

        default:
            printf("Invalid choice\n");
    }

    return 0;
}
```

```

/* Function to generate largest possible number */

void largestNumber(void)
{
    long long n, temp, result = 0;
    int freq[10] = {0};
    int digit, i;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    temp = n;

    while (temp > 0)
    {
        digit = temp % 10;
        freq[digit]++;
        temp = temp / 10;
    }

    for (i = 9; i ≥ 0; i--)
    {
        while (freq[i] > 0)
        {
            result = result * 10 + i;
            freq[i]--;
        }
    }

    printf("Largest possible number = %lld\n", result);
}

```

```

GNU nano 2.9.0
/* Function to generate smallest possible number */

void smallestNumber(void)
{
    long long n, temp, result = 0;
    int freq[10] = {0};
    int digit, i;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    temp = n;

    while (temp > 0)
    {
        digit = temp % 10;
        freq[digit]++;
        temp = temp / 10;
    }

    /* Find the first non-zero digit */
    for (i = 1; i <= 9; i++)
    {
        if (freq[i] > 0)
        {
            result = i;
            freq[i]--;

            /* Put zeroes after the first digit */
            while (freq[0] > 0)
            {
                result = result * 10;
                freq[0]--;
            }

            break;
        }
    }
}

```

GNU nano 9.0

```
/* Add remaining digits in ascending order */
for (i = 1; i ≤ 9; i++)
{
    while (freq[i] > 0)
    {
        result = result * 10 + i;
        freq[i]--;
    }

    printf("Smallest possible number = %lld\n", result);
}

/* Function to swap first and last digits */
void swapFirstLast(void)
{
    long long n;
    long long first, last;
    long long divisor = 1;
    long long middle, result;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n < 10)
    {
        printf("After swapping first and last digits = %lld\n", n);
        return;
    }

    last = n % 10;

    while (n / divisor ≥ 10)
    {
        divisor = divisor * 10;
    }

    first = n / divisor;
    middle = (n % divisor) / 10;

    result = last * divisor + middle * 10 + first;

    printf("After swapping first and last digits = %lld\n", result);
}
```

```
/* Function to check palindrome */  
  
void checkPalindrome(void)  
{  
    long long n, temp, reverse = 0;  
    int digit;  
  
    printf("Enter a positive integer: ");  
    scanf("%lld", &n);  
  
    temp = n;  
  
    while (temp > 0)  
    {  
        digit = temp % 10;  
        reverse = reverse * 10 + digit;  
        temp = temp / 10;  
    }  
  
    if (reverse == n)  
    {  
        printf("%lld is a palindrome number.\n", n);  
    }  
    else  
    {  
        printf("%lld is not a palindrome number.\n", n);  
    }  
}
```



```
(kali㉿kali)-[~]  
$ nano 17.c
```

```
(kali㉿kali)-[~]  
$ gcc 17.c -o 17
```

```
(kali㉿kali)-[~]  
$ ./17  
—— MENU ——  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
Enter your choice: 1  
Enter a positive integer: 31042  
Largest possible number = 43210
```

```
(kali㉿kali)-[~]  
$ ./17  
—— MENU ——  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
Enter your choice: 2  
Enter a positive integer: 31042  
Smallest possible number = 10234
```

```
(kali㉿kali)-[~]  
$ ./17  
—— MENU ——  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
Enter your choice: 3  
Enter a positive integer: 12345  
After swapping first and last digits = 52341
```

```
(kali㉿kali)-[~]  
$ ./17  
—— MENU ——  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
Enter your choice: 4  
Enter a positive integer: 12321  
12321 is a palindrome number.
```

```
(kali㉿kali)-[~]  
$ ./17  
----- MENU -----  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
Enter your choice: 4  
Enter a positive integer: 12321  
12321 is a palindrome number.  
  
(kali㉿kali)-[~]  
$ ./17  
----- MENU -----  
1. Generate largest possible number  
2. Generate smallest possible number  
3. Swap first and last digits  
4. Check palindrome  
Enter your choice: 4  
Enter a positive integer: 12345  
12345 is not a palindrome number.  
  
(kali㉿kali)-[~]  
$
```

4. Conclusion:

The program therefore satisfies the requirement of four separate void user-defined functions, called simply as the largest number, smallest number, swap first last, palindrome, without passing arguments.

NAME: TAANISH AGARWAL

BATCH: B-15

SAP ID: 590034662